DHBW
Duale Hochschule
Baden-Württemberg
Mannheim

THEMA PROJEKTARBEIT

Konzeption, Implementierung und Evaluation eines offlinefähigen Reise- und Sprachlern-Dashboards auf Basis eines Raspberry Pi

im Studiengang

Informatik

an der *Duale Hochschule Baden-Württemberg Mannheim*

von

Name, Vorname: Michael Kurz, Schayan Goudarzi

Abgabedatum: 12.04.2027

Bearbeitungszeitraum	01.10.2026 - 12.04.2027
Matrikelnummer, Kurs	6722742, 3174980 / TINF24IT1
Ausbildungsfirma	Logicalis GmbH, Alstom Transportation GmbH
Betrieblicher Betreuer	Jürgen Schultheis

Unterschrift Betreuer _____

Verantwortlichkeiten

Diese Arbeit ist eine Gemeinschaftsarbeit. Die folgende Übersicht zeigt die Hauptverantwortung je Bereich; innerhalb der Kapitel ist der jeweils verantwortliche Verfasser zusätzlich in Klammern gekennzeichnet.

Bereich	Verantwortlich
Systemarchitektur, Backend, Geschäftslogik, Security	Michael Kurz
Frontend, Datenbankanbindung, Raspberry Pi, Deployment	Schayan Goudarzi
Konzept, Marktanalyse, Umfrage, Evaluation, Dokumentation	gemeinsam

Ehrenwörtliche Erklärung

Gemäß § 20 der Studien- und Prüfungsordnung der Dualen Hochschule Baden-Württemberg

Hiermit versichern wir, Michael Kurz und Schayan Goudarzi, durch unsere Unterschriften, dass wir die vorliegende Arbeit mit dem Thema „Konzeption, Implementierung und Evaluation eines offlinefähigen Reise- und Sprachlern-Dashboards auf Basis eines Raspberry Pi“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet haben. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder unveröffentlichten Schriften oder dem Internet entnommen worden sind, sind als solche kenntlich gemacht. Wir versichern zudem, dass diese Arbeit bei keiner anderen Prüfung mit gleichem oder vergleichbarem Inhalt vorgelegt und bislang nicht veröffentlicht wurde und dass die eingereichte elektronische Fassung mit der gedruckten Ausfertigung übereinstimmt.

Wir haben bei der Erstellung der Arbeit KI-Werkzeuge eingesetzt. Dies haben wir an den entsprechenden Stellen der Arbeit kenntlich gemacht; eine Übersicht findet sich in der Anmerkung zur Nutzung von Künstlicher Intelligenz am Ende der Arbeit.

Mannheim, den _____

Michael Kurz	Schayan Goudarzi

Kurzfassung

Hier kommt die Kurzfassung (200 bis 300 Wörter): Fragestellung, methodisches Vorgehen und Hauptergebnisse.

Abstract

English abstract (200 to 300 words): research question, approach and main results.

Inhaltsverzeichnis

Verantwortlichkeiten 2

Ehrenwörtliche Erklärung 3

Kurzfassung 4

Abstract 5

1. Einleitung 11

1.1. Einordnung und Motivation . . . 11
1.2. Problemstellung . . . 11
1.3. Zielsetzung . . . 12
1.4. Aufgabenstellung . . . 12
1.5. Aufbau der Arbeit . . . 13

2. Grundlagen 13

2.1. Begriffliche Grundlagen . . . 14
2.1.1. Dashboard . . . 14
2.1.2. Offlinefähigkeit . . . 14
2.1.3. Lokale Datenhaltung und Datensparsamkeit . . . 14
2.1.4. Mehrbenutzerbetrieb auf einem Gerät . . . 15
2.2. Lernen von Vokabeln . . . 15
2.2.1. Vergessenskurve . . . 15
2.2.2. Verteilte Wiederholung und das Leitner-System . . . 15
2.2.3. Abrufeffekt . . . 16
2.2.4. Sprachniveau für Reisende . . . 16
2.2.5. Schriftsysteme der Zielsprachen und Umschrift . . . 16
2.3. Softwarequalität und Gebrauchstauglichkeit . . . 17
2.3.1. Qualitätsmodell nach ISO/IEC 25010 . . . 17
2.3.2. Gebrauchstauglichkeit nach ISO 9241-11 . . . 17
2.3.3. Heuristiken nach Nielsen . . . 18
2.3.4. Usability-Test mit Testnutzern . . . 18
2.3.5. System Usability Scale . . . 18
2.4. Anforderungsermittlung . . . 19
2.4.1. Requirements Engineering und Anforderungsarten . . . 19
2.4.2. Online-Fragebogen und Likert-Skala . . . 19
2.4.3. Stichprobe . . . 20
2.4.4. Priorisierung mit MoSCoW . . . 20
2.4.5. Use Cases und das Anwendungsfalldiagramm . . . 20

2.4.6. Entity-Relationship-Modell . . . 20
2.5. Webanwendungen . . . 21
2.5.1. Client-Server-Architektur und HTTP . . . 21
2.5.2. Model-View-Controller und serverseitiges Rendering . . . 21
2.5.3. Sitzungen und Cookies . . . 22
2.5.4. Schichtenarchitektur . . . 22
2.5.5. Wege zur Offlinefähigkeit . . . 23
2.5.6. Erreichbarkeit im lokalen Netz . . . 24
2.6. Verwendete Technologien . . . 24
2.6.1. Java . . . 24
2.6.2. Spring Boot . . . 24
2.6.3. Thymeleaf . . . 24
2.6.4. Bootstrap und WebJars . . . 25
2.6.5. H2 und Jakarta Persistence . . . 25
2.6.6. Raspberry Pi 5 und Raspberry Pi OS . . . 25
2.6.7. Maven und Git . . . 25

3. Marktanalyse 26
3.1. Vorgehen und Quellen . . . 26
3.2. Reiseplaner . . . 26
3.2.1. Wanderlog . . . 26
3.2.2. TripIt . . . 26
3.3. Sprachlern-Apps . . . 27
3.3.1. LingoDeer . . . 27
3.3.2. Anki . . . 27
3.4. Self-hosted-Lösungen . . . 27
3.4.1. AdventureLog . . . 27
3.5. Bewertungskriterien . . . 28
3.6. Vergleich . . . 28
3.7. Abgeleitete Lücke . . . 28

4. Anforderungsanalyse 30
4.1. Umfrage . . . 31
4.1.1. Entwicklung des Fragebogens . . . 31
4.1.2. Teilnehmer und Durchführung . . . 31
4.1.3. Ergebnisse der geschlossenen Fragen . . . 31
4.1.4. Freitextantworten . . . 31
4.1.5. Limitationen . . . 31
4.2. Use Cases . . . 31
4.2.1. Akteur . . . 31

4.2.2. Anwendungsfalldiagramm . . . 31
4.2.3. Beschreibung der Anwendungsfälle . . . 31
4.3. Funktionale Anforderungen . . . 31
4.3.1. Reisebereich . . . 31
4.3.2. Sprachbereich . . . 31
4.3.3. Benutzer und Profile . . . 31
4.4. Nichtfunktionale Anforderungen . . . 31
4.5. Priorisierung (MoSCoW) . . . 31

5. Konzeption **31**
5.1. Systemarchitektur . . . 32
5.1.1. Schichten der Anwendung . . . 32
5.1.2. Verteilung auf Raspberry Pi und Endgeräte . . . 32
5.1.3. Technologieentscheidungen . . . 32
5.2. Datenmodell . . . 32
5.2.1. ER-Modell . . . 32
5.2.2. Reisebereich . . . 32
5.2.3. Sprachbereich . . . 32
5.2.4. Lernfortschritt . . . 32
5.3. Oberfläche . . . 32
5.3.1. Navigationskonzept . . . 32
5.3.2. Reise-Screens . . . 32
5.3.3. Sprach-Screens . . . 32
5.3.4. Mockups . . . 32
5.4. Sicherheitskonzept . . . 32

6. Implementierung **32**
6.1. Projektstruktur und Build . . . 33
6.2. Backend . . . 33
6.2.1. Profilauswahl und Sitzung . . . 33
6.2.2. Reise-Services . . . 33
6.2.3. Wiederholungslogik . . . 33
6.2.4. Quiz . . . 33
6.3. Frontend . . . 33
6.3.1. Templates . . . 33
6.3.2. Bootstrap im Offline-Betrieb . . . 33
6.4. Datenhaltung . . . 33
6.4.1. Entities und Repositories . . . 33
6.4.2. H2-Datenbankdatei . . . 33
6.4.3. Laden der Sprachdaten . . . 33

6.5. Betrieb auf dem Raspberry Pi . . . 33
6.5.1. Betriebssystem und Java . . . 33
6.5.2. Dienst und Autostart . . . 33
6.5.3. Hostname und Erreichbarkeit . . . 33
6.6. Automatisierte Tests . . . 33

7. Evaluation **33**
7.1. Verifikation . . . 34
7.1.1. Unit-Tests . . . 34
7.1.2. Funktionstests je Anwendungsfall . . . 34
7.1.3. Datenbanktests . . . 34
7.1.4. Offline-Test . . . 34
7.1.5. Antwortzeiten . . . 34
7.2. Validierung . . . 34
7.2.1. Testdesign . . . 34
7.2.2. Teilnehmer . . . 34
7.2.3. Aufgaben . . . 34
7.2.4. Ergebnis des System Usability Scale . . . 34
7.2.5. Beobachtungen . . . 34
7.3. Abgleich mit den Anforderungen . . . 34

8. Kritische Reflexion **34**
8.1. Methodik . . . 34
8.2. Prototyp . . . 34
8.3. Vorgehen und Zusammenarbeit . . . 34

9. Ausblick **34**
9.1. Funktionale Erweiterungen . . . 35
9.2. Technische Erweiterungen . . . 35
9.3. Weitere Untersuchungen . . . 35

10. Methoden und Verfahren **35**
10.1. Vorgehen . . . 35
10.2. Eingesetzte Werkzeuge und Versionen . . . 35
10.3. Nachvollziehbarkeit . . . 35

Literatur **36**

Anmerkung zur Nutzung von Künstlicher Intelligenz **41**

A. Fragebogen der Online-Umfrage **42**

Abbildungsverzeichnis

2.1. Verteilte Wiederholung mit fünf Fächern nach Leitner, Abstände wie im Prototyp (eigene Darstellung) 16
2.2. Die neun Qualitätsmerkmale der ISO/IEC 25010, hervorgehoben die in dieser Arbeit verwendeten (eigene Darstellung nach [13, Abschn. 4.1]) . . . 17
2.3. Weg einer Anfrage durch die Schichten einer Spring-MVC-Anwendung (eigene Darstellung) 22
2.4. Zwei Wege zur Offlinefähigkeit: Logik und Daten im Browser (links) oder ein eigener Server im lokalen Netz (rechts); eigene Darstellung 23
3.1. Einordnung der betrachteten Systeme nach Funktionsumfang und Betriebsmodell (eigene Darstellung) 29

Tabellenverzeichnis

3.1. Kriteriengestützter Vergleich der betrachteten Systeme (Stand 2. November 2026) 29
10.1. Eingesetzte Werkzeuge und Bibliotheken mit Versionen (Stand 4. November 2026) 35
K1. Übersicht über die verwendeten KI-basierten Werkzeuge 41
B.1. Antworten auf die geschlossenen Fragen 43
B.2. Genutzte Apps und Freitextantworten (Rechtschreibung behutsam angeglichen, Inhalt unverändert) 44

1. Einleitung

1.1. Einordnung und Motivation

Im Jahr 2025 unternahmen 80,5 % der deutschsprachigen Bevölkerung ab 14 Jahren mindestens eine Urlaubsreise von fünf oder mehr Tagen, und 78 % dieser Reisen führten ins Ausland [1]. Vor Reiseantritt ist einiges vorzubereiten. Dazu gehören organisatorische Aufgaben wie das Planen der einzelnen Reisetage, das Erstellen einer Packliste und die Übersicht über das Budget, aber auch das Erlernen grundlegender Wörter und Redewendungen in der Sprache des Ziellandes.

Beide Bereiche hängen zusammen: Der Wunsch, ins Ausland zu reisen, zählt zu den stärksten praktischen Motiven für das Erlernen einer Fremdsprache, wie eine Befragung von 229 Studierenden zeigt [2], und umgekehrt erleichtern Sprachkenntnisse die Orientierung vor Ort. In der Praxis werden Reiseorganisation und Sprachenlernen dennoch mit getrennten Werkzeugen bearbeitet.

Für beide Bereiche existieren zahlreiche digitale Anwendungen; allein für die Urlaubsplanung nutzen Reisende im Schnitt vier verschiedene Apps [3]. Die Reiseplanung decken Dienste wie Wanderlog [4] oder TripIt [5] ab, für das Sprachenlernen stehen Applikationen wie Duolingo [6], Babbel [7] oder Anki [8] zur Verfügung. Diese Produkte konzentrieren sich jeweils nur auf einen Teilbereich, setzen in der Regel eine kontinuierliche Internetverbindung voraus und speichern Inhalte sowie persönliche Nutzerdaten in der Cloud, wie die Marktanalyse in Kapitel 3 zeigt. Wer eine Reise vorbereitet, muss seine Daten daher über mehrere Anbieter verteilen und gibt die Kontrolle darüber teilweise ab.

Gerade im Ausland ist eine stabile Internetverbindung aber nicht immer gegeben. Die Inlandspreise des EU-Roamings gelten nur innerhalb der EU; außerhalb dürfen die Anbieter weiterhin Aufschläge erheben, sodass insbesondere die mobile Datennutzung dort meist wesentlich teurer ist [9], [10]. Das betrifft alle drei in dieser Arbeit betrachteten Regionen. Unterwegs, wo die Apps am ehesten gebraucht werden, sind cloudgebundene Anwendungen damit nur eingeschränkt nutzbar.

Beide Bereiche werden somit bislang getrennt und überwiegend cloudgebunden abgedeckt. Daraus ergibt sich die im folgenden Abschnitt dargestellte Problemstellung.

1.2. Problemstellung

Die Vorbereitung einer Auslandsreise umfasst zwei Aufgabenbereiche, die bislang getrennt voneinander bearbeitet werden: a) die organisatorische Planung der Reise und b) die sprachliche Vorbereitung auf das jeweilige Zielland. Für beide Bereiche existieren zwar etablierte Anwendungen, eine integrierte Lösung, die beide Bereiche verbindet und dabei

ohne ständige Internetverbindung sowie ohne Auslagerung der persönlichen Daten an externe Anbieter auskommt, fehlt bislang jedoch.

Diese Studienarbeit untersucht daher, inwiefern sich ein offlinefähiges, lokal auf einem Raspberry Pi betriebenes Dashboard umsetzen lässt, das Reiseplanung und Sprachenlernen in einer einzigen Anwendung vereint. Diese Leitfrage wird in drei Teilfragen untergliedert. Zunächst ist zu klären, welche funktionalen und nichtfunktionalen Anforderungen potenzielle Nutzer an ein solches System stellen. Darauf aufbauend stellt sich die Frage, wie sich ein solches System technisch auf einem Raspberry Pi umsetzen lässt. Abschließend ist zu untersuchen, inwieweit der entwickelte Prototyp die ermittelten Anforderungen hinsichtlich Funktionalität und Benutzbarkeit erfüllt.

1.3. Zielsetzung

Die Arbeit zielt nicht darauf, bestehende Reiseplaner oder Sprachlern-Apps zu ersetzen, sondern beide Funktionen in einer lokal betriebenen, offlinefähigen Anwendung zusammenzuführen. Entwickelt wird ein Prototyp eines Reise- und Sprachlern-Dashboards auf Basis eines Raspberry Pi, der ohne Internetverbindung im lokalen Netzwerk läuft. Im Einzelnen soll die Anwendung die Planung einer Reise an einem Ort bündeln, also Reiseziele, Tagesplanung, Packlisten und Budget, und zugleich das Lernen der Zielsprache durch Vokabeln, Lektionen und Quiz unterstützen. Der Lernfortschritt wird dabei je Benutzer nachvollziehbar festgehalten. Die gesamte Anwendung soll vollständig offline und datensparsam auf lokaler Hardware funktionieren. Betrachtet werden exemplarisch die Regionen Russland, Japan und Korea. Als erfolgreich gilt die Umsetzung, wenn die aus der Problemstellung abgeleiteten Aufgaben nachvollziehbar bearbeitet und die zuvor erhobenen Anforderungen durch den Prototyp erfüllt werden.

1.4. Aufgabenstellung

Aus der zuvor beschriebenen Problemstellung ergeben sich die folgenden Aufgaben, die im weiteren Verlauf der Arbeit bearbeitet werden:

1. Grundlagen und Marktanalyse
   - 1.1. Vergleichbare Systeme analysieren (Reiseplaner, Sprachlern-Apps, lokale Lösungen)
   - 1.2. Bewertungskriterien aus der Literatur ableiten
   - 1.3. Die Marktlücke bestimmen
2. Anforderungsanalyse
   - 2.1. Nutzeranforderungen erheben (Umfrage)

   2.2. Funktionale und nichtfunktionale Anforderungen ableiten
   2.3. Anforderungen priorisieren (MoSCoW)
3. Konzeption
   3.1. Systemarchitektur entwerfen
   3.2. Datenmodell entwerfen
   3.3. Oberfläche konzipieren (Wireframes)
4. Implementierung
   4.1. Prototyp auf Basis von Spring Boot umsetzen
   4.2. Offline-Betrieb auf dem Raspberry Pi einrichten
5. Evaluation
   5.1. Verifikation durch Funktions- und Datenbanktests
   5.2. Validierung durch einen Usability-Test mit Testnutzern

1.5. Aufbau der Arbeit

Die Arbeit gliedert sich in zehn Kapitel. Kapitel 2 stellt die Grundlagen bereit. Kapitel 3 analysiert vergleichbare Systeme und leitet die Marktlücke ab, Kapitel 4 erhebt und priorisiert die Anforderungen. Kapitel 5 entwickelt das Konzept aus Architektur, Datenmodell und Oberfläche, Kapitel 6 schildert die Implementierung und den Betrieb auf dem Raspberry Pi. Kapitel 7 evaluiert das Ergebnis, Kapitel 8 reflektiert es kritisch und Kapitel 9 gibt einen Ausblick. Kapitel 10 fasst die eingesetzten Methoden und Werkzeuge zusammen.

2. Grundlagen

Dieses Kapitel stellt die Grundlagen bereit, auf denen die folgenden Kapitel aufbauen. Die Begriffe und Methoden stehen in den Abschnitten 2.1 bis 2.4, die technischen Grundlagen und die eingesetzten Werkzeuge in den Abschnitten 2.5 und 2.6. Hier geht es nur darum, was die Konzepte sind und woher sie stammen; wie sie in dieser Arbeit angewendet werden, zeigen die Kapitel 3 bis 7 jeweils mit Verweis hierher. Die Auswahl beschränkt sich auf das, was für das Verständnis der Marktanalyse, der Anforderungsanalyse, der Konzeption und der Evaluation notwendig ist.

2.1. Begriffliche Grundlagen

2.1.1. Dashboard

Wexler, Shaffer und Cotgreave definieren ein Dashboard als visuelle Darstellung von Daten, die dazu dient, Zustände zu überwachen oder Verständnis zu erleichtern, und merken an, dass es daneben zahlreiche engere Definitionen gibt [11, Einleitung]. Eine solche engere Fassung stammt von Few: Ein Dashboard ist für ihn die visuelle Darstellung der wichtigsten Informationen, die zum Erreichen eines oder mehrerer Ziele benötigt werden, so verdichtet und auf einem einzigen Bildschirm angeordnet, dass sie auf einen Blick überwacht werden können [12, S. 34]. Beide Definitionen kommen aus der Überwachung von Kennzahlen in Unternehmen. In dieser Arbeit wird der Begriff etwas weiter gefasst: Das Reise- und Sprachlern-Dashboard bündelt auf einer gemeinsamen Startseite nicht nur Informationen, etwa die nächste Reise und die heute fälligen Vokabeln, sondern auch den Einstieg in die Funktionen beider Bereiche. Der Kern der Definition bleibt erhalten: eine Seite, die ohne Suchen zeigt, was gerade ansteht.

2.1.2. Offlinefähigkeit

Eine Anwendung gilt in dieser Arbeit als offlinefähig, wenn sämtliche Funktionen ohne Verbindung zum Internet nutzbar sind. Das schließt nicht aus, dass die Anwendung über ein lokales Netzwerk erreicht wird; entscheidend ist, dass keine Komponente auf Dienste außerhalb dieses Netzwerks angewiesen ist. ISO/IEC 25010 nennt den Betrieb ohne Netzverbindung als einen Nutzungskontext, auf den ein Produkt ausgelegt sein kann [13, Abschn. 3.8] (zur technischen Umsetzung vgl. Abschnitt 2.5.5).

2.1.3. Lokale Datenhaltung und Datensparsamkeit

Von der Offlinefähigkeit zu unterscheiden ist die lokale Datenhaltung: Die Daten liegen auf einem Gerät unter Kontrolle der Nutzer und nicht auf den Servern eines Anbieters. Beide Eigenschaften treten getrennt voneinander auf, wie die Marktanalyse in Kapitel 3 zeigt. Lokale Datenhaltung ist zugleich die einfachste Form von Datensparsamkeit. Die Datenschutz-Grundverordnung verlangt in Artikel 5 Absatz 1, dass personenbezogene Daten dem Zweck angemessen und auf das notwendige Maß beschränkt sein müssen [14]. Für eine privat genutzte Anwendung im eigenen Haushalt gilt die Verordnung nach Artikel 2 Absatz 2 zwar nicht unmittelbar; der Grundsatz dient hier dennoch als Maßstab, weil Reisedaten und Lernstände persönliche Daten sind, die bei den in Kapitel 3 betrachteten Produkten beim Anbieter liegen.

2.1.4. Mehrbenutzerbetrieb auf einem Gerät

Läuft eine Anwendung auf einem Gerät, das mehrere Personen im selben Haushalt nutzen, müssen deren Daten voneinander getrennt werden: Die Reisen und Lernstände einer Person dürfen nicht in denen einer anderen auftauchen. Üblich ist dafür ein Benutzerkonto mit Passwort. In einem privaten WLAN hat das Passwort jedoch keinen Mehrwert, weil nur Personen im Haushalt das Gerät erreichen und der Schutz des Geräts selbst über den Zugang zum WLAN erfolgt. Diese Arbeit verwendet deshalb Profile ohne Passwort: Beim Start wählt die Person ihr Profil aus, und die Anwendung merkt sich die Wahl für die Dauer der Nutzung. Streaming-Dienste wie Netflix machen es mit ihrer Profilauswahl genauso. Technisch beruht das Verfahren auf Sitzungen (Abschnitt 2.5.3); die Einordnung als Sicherheitsmerkmal folgt in Abschnitt 2.3.1.

2.2. Lernen von Vokabeln

2.2.1. Vergessenskurve

Ebbinghaus untersuchte 1885 in Selbstversuchen mit sinnlosen Silben, wie viel von Gelerntem nach bestimmten Zeitabständen noch verfügbar ist. Er maß dazu, wie viel Aufwand das erneute Lernen bis zur fehlerfreien Wiedergabe im Vergleich zum ersten Lernen ersparte. Das Ergebnis ist die nach ihm benannte Vergessenskurve: Der Verlust ist unmittelbar nach dem Lernen am größten und verlangsamt sich danach deutlich [15]. Für das Vokabellernen folgt daraus, dass eine einmal gelernte Vokabel früh wiederholt werden muss, später aber immer seltener.

2.2.2. Verteilte Wiederholung und das Leitner-System

Ebbinghaus beobachtete außerdem, dass über mehrere Tage verteilte Wiederholungen wirksamer sind als dieselbe Zahl von Wiederholungen an einem Tag [15]. Dieser Abstandseffekt hält auch moderner Prüfung stand: Cepeda et al. fassen in einer Meta-Analyse Studien zum verbalen Erinnern zusammen und zeigen, dass verteiltes Lernen dem geballten Lernen über verschiedene Abstände und Behaltensintervalle hinweg überlegen ist [16].

Leitner übertrug das Prinzip 1972 in ein praktisches Verfahren mit Karteikarten, das er selbst als „Lernkartei“ bezeichnet [17, S. 64]: Die Karten wandern durch fünf Fächer, eine richtig beantwortete Karte rückt ein Fach weiter und wird seltener abgefragt, eine falsch beantwortete fällt in das erste Fach zurück. Karten, die das fünfte Fach fehlerfrei durchlaufen haben, gelten als gelernt und werden aussortiert [17, S. 71]. Anwendungen wie Anki setzen dieses Prinzip um (Kapitel 3). Der Prototyp verwendet eine Variante mit fünf Fächern und festen Abständen, die Abbildung 2.1 zeigt und die in Kapitel 6 beschrieben wird.

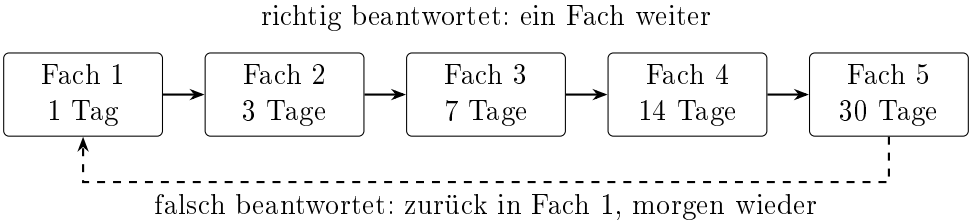


Abbildung 2.1.: Verteilte Wiederholung mit fünf Fächern nach Leitner, Abstände wie im Prototyp (eigene Darstellung)

2.2.3. Abrufeffekt

Neben dem Abstand spielt die Art der Wiederholung eine Rolle. Karpicke und Roediger zeigen, dass das aktive Abrufen aus dem Gedächtnis, wie es eine Karteikartenabfrage verlangt, mehr zum langfristigen Behalten beiträgt als wiederholtes Lesen desselben Stoffs [18]. Beides zusammen spricht dafür, Vokabeln abzufragen statt nur anzuzeigen, und die Abfragen über die Zeit zu verteilen.

2.2.4. Sprachniveau für Reisende

Wie viel Sprache eine Reisende oder ein Reisender überhaupt braucht, lässt sich mit dem Gemeinsamen Europäischen Referenzrahmen für Sprachen einordnen. Er beschreibt sechs Niveaustufen von A1 bis C2. Auf der Stufe A1 kann eine Person vertraute, alltägliche Ausdrücke und einfache Sätze verstehen und verwenden, sich und andere vorstellen und auf einfache Fragen zur Person antworten; auf A2 kommen Sätze und häufig gebrauchte Ausdrücke zu unmittelbar relevanten Bereichen wie Einkaufen und Wegbeschreibungen hinzu [19]. Für die Vorbereitung einer Reise ist damit der Bereich A1 bis A2 relevant: Begrüßung, Zahlen, Bestellen, Fragen nach dem Weg.

2.2.5. Schriftsysteme der Zielsprachen und Umschrift

Die drei Zielsprachen der Arbeit verwenden keine lateinische Schrift. Russisch wird in kyrillischer Schrift geschrieben, einem Alphabet mit 33 Buchstaben. Japanisch kombiniert die beiden Silbenschriften Hiragana und Katakana mit den aus dem Chinesischen übernommenen Kanji. Koreanisch verwendet Hangul, ein Alphabet, dessen Buchstaben zu Silbenblöcken zusammengesetzt werden. Für Lernende, die diese Schriften noch nicht lesen können, braucht jede Vokabel eine Umschrift in lateinische Buchstaben. Dafür gibt es je Sprache etablierte Systeme: für das Kyrillische die Transliteration nach ISO 9 [20], für das Japanische die modifizierte Hepburn-Umschrift, wie sie etwa die Library of Congress verwendet [21], und für das Koreanische die Revidierte Romanisierung, die das südkoreanische Nationale Institut für die koreanische Sprache festgelegt hat [22]. Welches System der Prototyp je Sprache verwendet, legt das Datenmodell in Kapitel 5 fest.

2.3. Softwarequalität und Gebrauchstauglichkeit

2.3.1. Qualitätsmodell nach ISO/IEC 25010

Die Norm ISO/IEC 25010 definiert ein Qualitätsmodell für Softwareprodukte, das die Produktqualität in neun Merkmale gliedert: funktionale Eignung, Leistungseffizienz, Kompatibilität, Interaktionsfähigkeit, Zuverlässigkeit, Sicherheit, Wartbarkeit, Flexibilität und Betriebssicherheit; jedes Merkmal ist in Teilmerkmale unterteilt [13, Abschn. 4.1]. Abbildung 2.2 zeigt die neun Merkmale; die vier für diese Arbeit relevanten sind hervorgehoben.

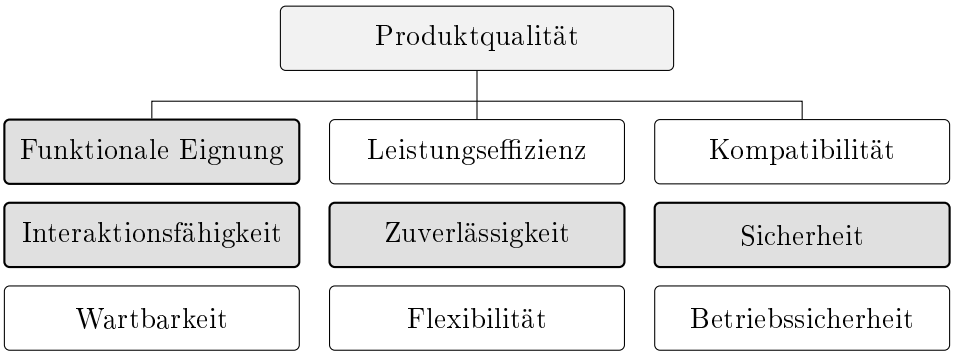


Abbildung 2.2.: Die neun Qualitätsmerkmale der ISO/IEC 25010, hervorgehoben die in dieser Arbeit verwendeten (eigene Darstellung nach [13, Abschn. 4.1])

Die funktionale Eignung beschreibt, ob ein Produkt Funktionen bereitstellt, die den Bedürfnissen der vorgesehenen Nutzer entsprechen, mit den Teilmerkmalen Vollständigkeit, Korrektheit und Angemessenheit. Die Zuverlässigkeit umfasst unter anderem die Verfügbarkeit, also die Fähigkeit, betriebsbereit und erreichbar zu sein, wenn das Produkt benötigt wird. Zur Sicherheit gehört die Vertraulichkeit, also dass Daten nur den dazu Berechtigten zugänglich sind; für den Mehrbenutzerbetrieb aus Abschnitt 2.1.4 heißt das, dass die Trennung der Profile ein Sicherheitsmerkmal ist, auch wenn sie ohne Passwort auskommt. Die Interaktionsfähigkeit, in der Vorgängerfassung von 2011 noch als Usability bezeichnet, fasst Teilmerkmale wie Erlernbarkeit, Bedienbarkeit und Schutz vor Benutzerfehlern zusammen [13, Abschn. 3].

Die Norm betont, dass die Gewichtung der Merkmale vom Nutzungskontext abhängt, also von Nutzern, Zielen, Umgebung und technischem Kontext [13, Abschn. 4.2]. Das Modell dient in Kapitel 3 als Grundlage der Vergleichskriterien und in Kapitel 4 zur Gliederung der nichtfunktionalen Anforderungen.

2.3.2. Gebrauchstauglichkeit nach ISO 9241-11

Die Gebrauchstauglichkeit (Usability) wird in DIN EN ISO 9241-11 definiert als das Ausmaß, in dem ein System durch bestimmte Benutzer in einem bestimmten Nutzungskontext genutzt werden kann, um bestimmte Ziele effektiv, effizient und zufriedenstellend zu er-

reichen [23, Abschn. 3.1.1]. Die drei Komponenten sind dort einzeln definiert: Effektivität als Genauigkeit und Vollständigkeit, mit denen Benutzer ihre Ziele erreichen, Effizienz als Verhältnis der eingesetzten Ressourcen, etwa Zeit und Aufwand, zu den erreichten Ergebnissen, und Zufriedenstellung als Ausmaß, in dem die Reaktionen der Benutzer ihren Erfordernissen und Erwartungen entsprechen [23, Abschn. 3.1.12 bis 3.1.14]. Der Nutzungskontext ist die Kombination aus Benutzern, Zielen, Aufgaben, Ressourcen und Umgebung [23, Abschn. 3.1.15]. Während ISO/IEC 25010 damit Eigenschaften des Produkts beschreibt, bezieht sich die Gebrauchstauglichkeit auf die tatsächliche Nutzung und lässt sich nur mit Nutzern messen (Abschnitte 2.3.4 und 2.3.5).

2.3.3. Heuristiken nach Nielsen

Ergänzend zu den Normen werden die zehn Usability-Heuristiken von Nielsen herangezogen, ein Satz von Faustregeln für die Gestaltung von Benutzeroberflächen, darunter die Sichtbarkeit des Systemstatus, die Übereinstimmung zwischen System und Lebenswelt der Nutzer, Konsistenz, Fehlervermeidung und die Bevorzugung von Wiedererkennen gegenüber Erinnern. Nielsen und Molich haben die heuristische Evaluation 1990 als Verfahren vorgestellt, bei dem wenige Prüfende eine Oberfläche anhand solcher Regeln durchgehen [24]; die heute verbreitete Fassung der zehn Heuristiken stammt von Nielsen [25]. Sie sind keine Norm, werden in Kapitel 5 aber als Prüfliste für den Entwurf der Oberfläche genutzt.

2.3.4. Usability-Test mit Testnutzern

Ein Usability-Test lässt Personen aus der Zielgruppe typische Aufgaben mit dem System bearbeiten, während beobachtet wird, wo sie stocken oder Fehler machen. Die Beobachtungen liefern die Effektivität (wurde die Aufgabe gelöst) und die Effizienz (wie lange hat es gedauert) im Sinne von Abschnitt 2.3.2. Nielsen und Landauer haben modelliert, wie viele Usability-Probleme mit einer gegebenen Zahl von Testpersonen gefunden werden, und daraus abgeleitet, dass bereits wenige Personen den Großteil der Probleme aufdecken, weil weitere Personen überwiegend bereits bekannte Probleme wiederholen [26]. Nielsen empfiehlt auf dieser Grundlage Tests mit etwa fünf Personen und, wenn Mittel vorhanden sind, mehrere kleine Testrunden statt einer großen [27]. Für einen studentischen Prototyp ist das ein realistischer Rahmen. Die Validierung in Kapitel 7 folgt diesem Vorgehen: Testnutzer bearbeiten vorgegebene Aufgaben, Aufgabenerfolg und benötigte Zeit werden festgehalten.

2.3.5. System Usability Scale

Die dritte Komponente der Gebrauchstauglichkeit, die Zufriedenstellung, wird üblicherweise mit einem Fragebogen erhoben. Der System Usability Scale (SUS) von Brooke be-

steht aus zehn Aussagen, die abwechselnd positiv und negativ formuliert sind und auf einer fünfstufigen Skala von „stimme überhaupt nicht zu“ bis „stimme voll zu“ bewertet werden; die Antworten werden zu einem Wert zwischen 0 und 100 verrechnet [28]. Der Fragebogen ist kurz und unabhängig von der Art des Systems, weshalb er sich für einen Vergleich mit anderen Studien eignet. Bangor, Kortum und Miller haben den SUS an einer großen Zahl von Studien ausgewertet und einen Mittelwert von rund 70 ermittelt; sie schlagen außerdem vor, die Punktwerte mit Adjektiven von „schlecht“ bis „exzellent“ zu beschreiben, damit ein einzelner Wert eingeordnet werden kann [29]. Für den Prototyp liefert das in Kapitel 7 einen Wert, der sich mit diesen Daten vergleichen lässt.

2.4. Anforderungsermittlung

2.4.1. Requirements Engineering und Anforderungsarten

Das Requirements Engineering umfasst nach Herrmann das Ermitteln, das Dokumentieren, das Prüfen und Abstimmen sowie das Verwalten von Anforderungen [30, Abschn. 1.3]. Bei den Anforderungen an das Produkt werden funktionale Anforderungen, die festlegen, was das System tun soll, von nichtfunktionalen Anforderungen unterschieden, die das Wie beschreiben, etwa wie schnell oder wie sicher; die nichtfunktionalen Anforderungen gliedern sich weiter in Qualitätsanforderungen und Randbedingungen [30, Abschn. 5.4]. Für die Qualitätsanforderungen empfiehlt Herrmann, die Qualitätsmerkmale der ISO/IEC 25010 aus Abschnitt 2.3.1 als Ausgangspunkt zu nehmen und sich auf die kritischen zu beschränken [30, Abschn. 5.7.4]; so wird in Kapitel 4 verfahren.

2.4.2. Online-Fragebogen und Likert-Skala

Als Erhebungstechnik kommt in dieser Arbeit eine quantitative Online-Befragung mit geschlossenen Fragen zum Einsatz. Geschlossene Fragen geben die Antwortmöglichkeiten vor und lassen sich dadurch direkt auszählen und vergleichen; offene Fragen liefern dagegen Begründungen und Wünsche in den Worten der Befragten und ergänzen die Zahlen. Für die Bewertung einzelner Funktionen wird eine fünfstufige Skala verwendet, die auf Likert zurückgeht: Befragte stufen ihre Zustimmung zu einer Aussage oder, wie hier, die Wichtigkeit eines Merkmals auf einer symmetrischen Skala mit neutraler Mitte ein [31]. Die Stufen sind geordnet, aber nicht notwendig gleich weit voneinander entfernt; Mittelwerte über solche Skalen sind deshalb eine Vereinfachung, die für eine Rangfolge ausreicht. Der SUS aus Abschnitt 2.3.5 verwendet dieselbe Skalenform.

2.4.3. Stichprobe

Die Teilnehmenden einer Befragung können selten zufällig aus der Zielgruppe gezogen werden. Werden stattdessen Personen einbezogen, die gerade zur Verfügung stehen oder leicht zugänglich sind, etwa über persönliche Kontakte oder eine öffentliche Online-Umfrage, spricht man von einer Gelegenheitsstichprobe. Das Schneeballverfahren geht einen Schritt weiter: Teilnehmende werden gebeten, die Befragung in ihren eigenen Netzwerken weiterzugeben. Beide Verfahren sind nach Döring und Bortz nicht-probabilistisch und können keine Repräsentativität beanspruchen; Rückschlüsse sind nur auf die Gesamtheit möglich, aus der die Stichprobe tatsächlich stammt [32, Abschn. 9.3.1]. Die Umfrage in Kapitel 4 beschreibt deshalb nur den befragten Personenkreis.

2.4.4. Priorisierung mit MoSCoW

Zur Priorisierung der ermittelten Anforderungen dient die MoSCoW-Methode aus dem agilen Vorgehensmodell DSDM. Sie ordnet jede Anforderung einer von vier Klassen zu: Must Have (ohne diese Anforderung ist die Lösung nicht einsetzbar), Should Have (wichtig, aber mit Umweg verzichtbar), Could Have (wünschenswert, wird bei Zeitnot als erstes gestrichen) und Won't Have this time (für diesen Zeitraum bewusst ausgeschlossen) [33]. DSDM setzt Zeit und Kosten eines Projekts fest und verhandelt stattdessen den Funktionsumfang; die Could-Klasse bildet dabei den Puffer. Das passt zu einem Prototyp mit festem Abgabetermin, weil die Methode den verbindlichen Kern von den Erweiterungen trennt und mit der Won't-Klasse auch festhält, was nicht gebaut wird.

2.4.5. Use Cases und das Anwendungsfalldiagramm

Ein Use Case (Anwendungsfall) beschreibt nach der UML-Spezifikation der Object Management Group eine Menge von Aktionen, die ein System für einen Akteur ausführt und die zu einem für diesen Akteur beobachtbaren Ergebnis führen; ein Akteur ist eine Rolle, die eine Person oder ein anderes System gegenüber dem betrachteten System einnimmt [34]. Das Anwendungsfalldiagramm stellt Akteure als Strichmännchen, Anwendungsfälle als Ellipsen und die Beteiligung als Verbindungslinien dar. Es zeigt, welche Funktionen ein System nach außen anbietet; den internen Ablauf lässt es offen. Für die Details eines Anwendungsfalls wird deshalb zusätzlich eine textuelle Beschreibung mit Vorbedingung, Ablauf und Nachbedingung verwendet.

2.4.6. Entity-Relationship-Modell

Das Entity-Relationship-Modell geht auf Chen zurück und beschreibt einen Ausschnitt der Realität durch Entitätstypen, ihre Attribute und die Beziehungen zwischen ihnen [35].

Eine Entität ist ein unterscheidbares Ding wie eine Reise; ihre Attribute sind Eigenschaften wie Ziel und Zeitraum. Beziehungen verbinden Entitäten, zum Beispiel eine Reise mit dem Benutzer, der sie angelegt hat, und tragen Kardinalitäten, die angeben, wie viele Entitäten auf jeder Seite beteiligt sein können (ein Benutzer, beliebig viele Reisen). Für die grafische Darstellung haben sich mehrere Notationen etabliert; in dieser Arbeit wird die Krähenfußnotation verwendet, bei der die Kardinalität am Linienende durch eine Gabel für „viele“ und einen Querstrich für „eins“ dargestellt wird. Das ER-Modell ist die Vorstufe des relationalen Datenbankschemas: Jeder Entitätstyp wird eine Tabelle mit den Attributen als Spalten, Beziehungen landen als Fremdschlüssel. Das Datenmodell des Prototyps wird in Kapitel 5 in dieser Form entworfen und in Kapitel 6 in Tabellen überführt.

2.5. Webanwendungen

2.5.1. Client-Server-Architektur und HTTP

Eine Webanwendung besteht aus einem Server, der die Anwendungslogik ausführt und Daten speichert, und einem Client, in der Regel ein Browser, der Seiten vom Server anfordert und darstellt. Die Kommunikation läuft über das Hypertext Transfer Protocol (HTTP): Der Client sendet eine Anfrage mit einer Methode und einem Ziel, der Server antwortet mit einem Statuscode und einem Inhalt. Die Methode GET fordert eine Ressource an, POST übermittelt Daten, etwa ein ausgefülltes Formular [36]. HTTP ist zustandslos: Jede Anfrage wird für sich behandelt, der Server weiß von sich aus nicht, ob zwei Anfragen von derselben Person stammen. Für eine Anwendung mit Benutzerprofilen ist das eine Lücke, die Abschnitt 2.5.3 schließt.

2.5.2. Model-View-Controller und serverseitiges Rendering

Das Model-View-Controller-Muster (MVC) trennt eine Anwendung in drei Teile: das Modell hält die Daten und die Logik, die View stellt sie dar, der Controller nimmt Eingaben entgegen und vermittelt zwischen beiden. Spring Web MVC setzt dieses Muster für Webanwendungen um: Ein zentraler Dispatcher nimmt jede HTTP-Anfrage entgegen, leitet sie an den zuständigen Controller weiter, dieser füllt ein Modell mit Daten und benennt eine View, und eine Template-Engine erzeugt daraus die HTML-Antwort [37]. Weil die fertige Seite auf dem Server entsteht, spricht man von serverseitigem Rendering. Der Browser erhält vollständiges HTML und benötigt keine eigene Anwendungslogik. Die Alternative, bei der der Browser nur Rohdaten erhält und die Seite selbst mit JavaScript aufbaut, verlagert Logik auf das Endgerät (dazu Abschnitt 2.5.5).

2.5.3. Sitzungen und Cookies

Um über mehrere zustandslose HTTP-Anfragen hinweg zu erkennen, dass sie von derselben Person stammen, verwendet der Server Sitzungen. Beim ersten Kontakt legt er eine Sitzung mit einer zufälligen Kennung an und schickt diese Kennung als Cookie an den Browser; der Browser sendet das Cookie bei jeder weiteren Anfrage an denselben Server mit, und der Server ordnet die Anfrage der Sitzung zu. Das Cookie-Verfahren ist in RFC 6265 beschrieben, einschließlich der Regeln, wann ein Browser ein Cookie speichert und mitsendet [38]. In der Sitzung kann der Server beliebige Daten ablegen, etwa das gewählte Benutzerprofil. Damit wird der passwortlose Mehrbenutzerbetrieb aus Abschnitt 2.1.4 technisch möglich: Die Profilwahl wird in der Sitzung gespeichert, und jede folgende Anfrage wird diesem Profil zugeordnet. Die Sitzungskennung ist dabei das einzige Geheimnis; sie sollte ausschließlich im Cookie und nicht in der Adresszeile übertragen werden, weil Adressen in Verläufen und Protokollen landen.

2.5.4. Schichtenarchitektur

Innerhalb des Servers hat sich eine Trennung in Schichten bewährt, die über das MVC-Muster hinausgeht: Controller nehmen Anfragen entgegen und geben Antworten zurück, Services enthalten die Geschäftslogik, Repositories kapseln den Zugriff auf die Datenbank. Spring unterstützt diese Trennung mit den Stereotyp-Annotationen `@Controller`, `@Service` und `@Repository`, die eine Klasse als Bestandteil der jeweiligen Schicht kennzeichnen und von Spring verwaltet werden [39]. Abbildung 2.3 zeigt den Weg einer Anfrage durch diese Schichten. Der Nutzen liegt in der Austauschbarkeit und Testbarkeit: Die Geschäftslogik in einem Service lässt sich ohne Browser und ohne Datenbank prüfen, und die Oberfläche kann geändert werden, ohne die Logik anzufassen. Für diese Arbeit ist die Trennung auch organisatorisch wichtig, weil die beiden Verfasser unterschiedliche Schichten verantworten (siehe die Übersicht der Verantwortlichkeiten am Anfang der Arbeit).

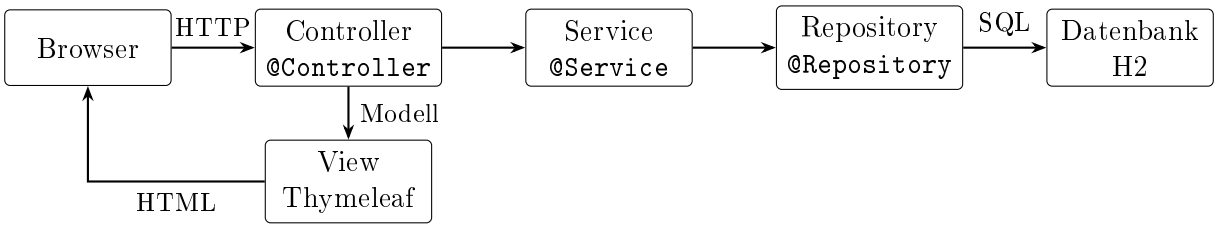


Abbildung 2.3.: Weg einer Anfrage durch die Schichten einer Spring-MVC-Anwendung (eigene Darstellung)

2.5.5. Wege zur Offlinefähigkeit

Fällt die Verbindung zwischen Browser und Server aus, ist eine klassische Webanwendung nicht mehr nutzbar. Um sie im Sinne von Abschnitt 2.1.2 offlinefähig zu machen, gibt es zwei grundsätzliche Ansätze, die sich darin unterscheiden, wo Logik und Daten liegen. Abbildung 2.4 stellt beide gegenüber.

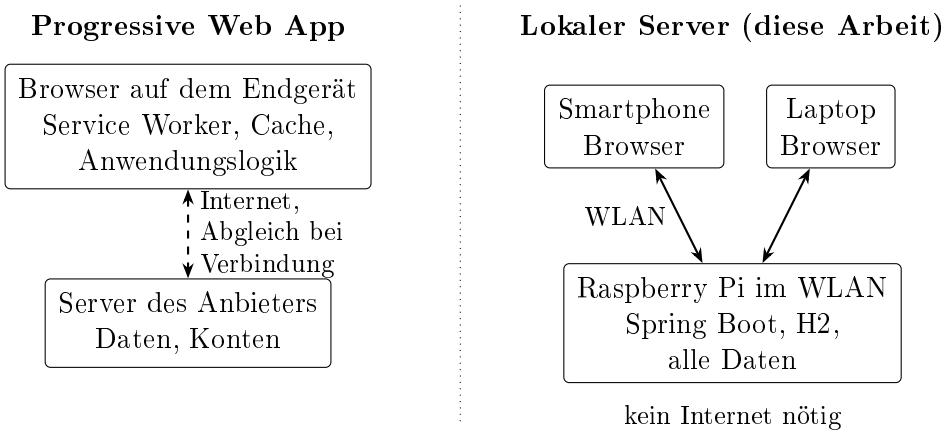


Abbildung 2.4.: Zwei Wege zur Offlinefähigkeit: Logik und Daten im Browser (links) oder ein eigener Server im lokalen Netz (rechts); eigene Darstellung

Progressive Web Apps (PWA) verlagern Logik und Daten in den Browser: Ein Service Worker, ein Skript im Hintergrund des Browsers, fängt Anfragen ab, liefert zuvor zwischengespeicherte Seiten und Daten auch ohne Netzverbindung aus und gleicht Änderungen bei der nächsten Verbindung mit dem Server ab; Ater beschreibt ihn als Stellvertreter zwischen Seite und Netz, der jede Anfrage abfangen, verändern oder selbst beantworten kann [40], [41, Kap. 2]. Die Anwendungslogik muss dafür in JavaScript im Browser laufen, der Offline-Stand gilt nur für das jeweilige Gerät, und die Daten liegen weiterhin beim Anbieter. Hinzu kommt eine Einschränkung, die für ein Heimnetz ins Gewicht fällt: Ein Service Worker darf nur auf Seiten registriert werden, die über HTTPS ausgeliefert werden, mit Ausnahme von localhost [40, Kap. 2]. Ein Server im WLAN ohne gültiges Zertifikat kann diesen Weg also nicht nutzen. Nach demselben Muster wie PWAs arbeiten auch die nativen Apps aus Kapitel 3.

Der Weg dieser Arbeit lässt die Architektur aus Abschnitt 2.5.1 unverändert und stellt stattdessen den Server in das lokale Netz der Nutzer. Die Anwendung läuft auf dem Raspberry Pi, alle Geräte im selben WLAN erreichen sie im Browser, und die Daten bleiben auf dem Pi. Mehrere Benutzer teilen sich dieselbe Anwendung und denselben Datenbestand; serverseitiges Rendering reicht aus, weil der Browser nichts selbst berechnen muss. Erreichbar ist die Anwendung allerdings nur, wo der Pi erreichbar ist: zu Hause bei der Reisevorbereitung ohne Einschränkung, unterwegs nur mit mitgeführtem Pi und einem lokalen Netz wie dem Hotspot eines Smartphones. Ausschlaggebend für diese Wahl waren die Datenhoheit der Nutzer, die gemeinsame Nutzung durch mehrere Profile

und der einfache Client, der sich mit dem Stapel aus Abschnitt 2.6 umsetzen lässt. Die Einschränkung der Erreichbarkeit wird in Kapitel 8 kritisch reflektiert.

2.5.6. Erreichbarkeit im lokalen Netz

Ein Server im Heimnetz hat eine IP-Adresse, die der Router vergibt und die sich ändern kann. Nutzer sollen diese Adresse nicht kennen müssen. Multicast DNS (mDNS) löst das Problem ohne zentralen Nameserver: Geräte im selben Netz beantworten Namensanfragen für die Endung `.local` selbst, indem sie auf eine Multicast-Anfrage mit ihrer aktuellen Adresse antworten [42]. Raspberry Pi OS unterstützt das ab Werk; der Pi ist im Netz unter seinem Hostnamen mit der Endung `.local` erreichbar, etwa `raspberrypi.local` [43]. Im Browser genügt dann dieser Name; die Einrichtung gehört zu Kapitel 6.

2.6. Verwendete Technologien

2.6.1. Java

Java ist eine objektorientierte, plattformunabhängige Programmiersprache: Der Compiler erzeugt Bytecode, der von der Java Virtual Machine auf jedem unterstützten System ausgeführt wird, auch auf der ARM-Architektur des Raspberry Pi. Verwendet wird Java 21, eine Version mit Langzeitunterstützung, die im September 2023 im OpenJDK-Projekt veröffentlicht wurde [44].

2.6.2. Spring Boot

Spring Boot ist ein Framework auf Basis des Spring Frameworks, das eine Webanwendung samt eingebettetem Webserver als einzelne ausführbare Datei verpackt und durch Voreinstellungen den Konfigurationsaufwand gering hält [45]. Es bringt Spring Web MVC für das Muster aus Abschnitt 2.5.2, die Stereotypen aus Abschnitt 2.5.4 und die Sitzungsverwaltung aus Abschnitt 2.5.3 mit. Auf dem Raspberry Pi genügt damit eine Java-Installation und eine Datei, um die Anwendung zu starten; ein separater Webserver ist nicht nötig.

2.6.3. Thymeleaf

Thymeleaf ist eine Template-Engine für serverseitiges Rendering: HTML-Vorlagen enthalten Platzhalter und Anweisungen in eigenen Attributen, die auf dem Server mit den Daten aus dem Modell gefüllt werden, sodass der Browser fertige Seiten erhält [46]. Die Vorlagen bleiben gültiges HTML und lassen sich auch ohne Server im Browser öffnen, was die Abstimmung zwischen Oberfläche und Logik erleichtert. Spring Boot bindet Thymeleaf als Standard-View-Technologie ein.

2.6.4. Bootstrap und WebJars

Bootstrap ist eine Sammlung von CSS-Klassen und JavaScript-Komponenten für responsive Oberflächen, die sich an die Bildschirmbreite anpassen [47]. Üblicherweise werden Bootstrap und seine Icons von einem Content Delivery Network im Internet geladen. Für einen Offline-Betrieb scheidet das aus. WebJars verpacken Web-Bibliotheken als Java-Archive, die über Maven wie jede andere Abhängigkeit in die Anwendung eingebunden und von dort ausgeliefert werden [48]. Bootstrap und die Bootstrap Icons liegen so in der ausführbaren Datei und stehen ohne Internetverbindung zur Verfügung.

2.6.5. H2 und Jakarta Persistence

H2 ist eine relationale Datenbank, die in Java geschrieben ist und im eingebetteten Modus innerhalb der Anwendung läuft; ihre Daten legt sie in einer lokalen Datei ab [49]. Ein eigener Datenbankserver entfällt, was auf dem Raspberry Pi Ressourcen spart und die Installation vereinfacht. Der Zugriff aus Java erfolgt über Jakarta Persistence, eine Spezifikation für die objekt-relationale Abbildung: Java-Klassen werden mit Annotationen als Entitäten gekennzeichnet, ihre Felder werden auf Tabellenspalten und ihre Verweise auf Fremdschlüssel abgebildet [50]. Hibernate ist die Implementierung dieser Spezifikation, die Spring Boot standardmäßig verwendet; sie erzeugt aus den Entitätsklassen das Tabellenschema und übersetzt Objektzugriffe in SQL [51]. Die Entitäten entsprechen damit direkt dem ER-Modell aus Abschnitt 2.4.6.

2.6.6. Raspberry Pi 5 und Raspberry Pi OS

Der Raspberry Pi 5 ist ein Einplatinencomputer mit einem 64-Bit-Vierkernprozessor (Arm Cortex-A76, 2,4 GHz), wahlweise 1 bis 16 GB Arbeitsspeicher, Gigabit-Ethernet und WLAN nach 802.11ac [52]. Für diese Arbeit wird die Variante mit 4 GB eingesetzt. Als Betriebssystem dient Raspberry Pi OS, eine Debian-basierte Linux-Distribution, die ohne grafische Oberfläche betrieben und über das Netzwerk verwaltet werden kann [43]. Der Pi übernimmt im Prototyp die Rolle des lokalen Servers aus Abschnitt 2.5.5.

2.6.7. Maven und Git

Maven löst die Abhängigkeiten des Projekts auf, kompiliert, testet und erzeugt die ausführbare Datei; die Versionen aller Abhängigkeiten stehen in einer zentralen Projektdatei, sodass der Build überall dasselbe Ergebnis liefert [53]. Git verwaltet die Versionen von Code und Dokumentation in einem gemeinsamen Repository der beiden Verfasser [54].

Kapitel 3 untersucht auf dieser Grundlage, welche vergleichbaren Systeme es am Markt gibt und welche Lücke sie offenlassen.

3. Marktanalyse

Dieses Kapitel bearbeitet die Aufgaben 1.1 bis 1.3: Es stellt vergleichbare Systeme vor, bewertet sie anhand von Kriterien, die aus dem Qualitätsmodell in Abschnitt 2.3.1 abgeleitet sind, und bestimmt daraus die Marktlücke, die der Prototyp schließen soll. Abschnitt 3.1 beschreibt das Vorgehen, die Abschnitte 3.2 bis 3.4 die betrachteten Systeme nach Gruppen, Abschnitt 3.5 die Kriterien, Abschnitt 3.6 den Vergleich und Abschnitt 3.7 die Lücke. Die Marktanalyse wurde gemeinsam bearbeitet.

3.1. Vorgehen und Quellen

Die Analyse stützt sich auf die öffentlich zugängliche Produktdokumentation, die Preisseiten der Anbieter und die Beschreibungen in den App-Stores (Stand: 2. November 2026). Ein eigener Praxistest der Anwendungen wurde nicht durchgeführt; die Aussagen beziehen sich daher auf die von den Herstellern dokumentierten Funktionen. Betrachtet werden kommerzielle Reiseplaner, kommerzielle Sprachlern-Apps und quelloffene Anwendungen, die auf eigener Hardware betrieben werden (self-hosted). Aus jeder Gruppe wurden verbreitete Vertreter ausgewählt, deren Funktionsumfang sich mit dem geplanten Dashboard überschneidet. Die Umfrage in Kapitel 4 bestätigt die Auswahl im Nachhinein: Von den dort genannten Apps sind Wanderlog, Anki und Duolingo vertreten, letzteres wird in Abschnitt 3.3 eingeordnet.

3.2. Reiseplaner

3.2.1. Wanderlog

Wanderlog ist ein Reiseplaner für Web und Mobilgeräte, der Reiseziele, Tagespläne und Buchungen in einer gemeinsamen Karten- und Listenansicht zusammenführt und zusätzlich eine Budgetverfolgung anbietet. Die Grundversion ist kostenlos; der Offline-Zugriff auf die Reisepläne, die Routenoptimierung und der Export nach Google Maps sind der Pro-Version vorbehalten, die für 39,99 USD pro Jahr angeboten wird [55]. Vom Funktionsumfang her ist Wanderlog das Produkt, das dem Reisebereich des Prototyps am nächsten kommt.

3.2.2. TripIt

TripIt ist kein Planungswerkzeug im engeren Sinn, Packlisten oder ein Budget gibt es dort nicht. Die Anwendung erzeugt aus weitergeleiteten Buchungsbestätigungen automatisch einen Reiseplan und dient damit der Organisation bereits gebuchter Reisen. Der

Reiseplan ist laut Hersteller auch ohne Internetverbindung abrufbar; Zusatzfunktionen wie Flugstatusmeldungen sind Teil des Abonnements TripIt Pro, das im deutschen App Store 48,99 EUR pro Jahr kostet [56]. TripIt steht hier für den Typ Reiseorganisator, bei dem die Daten aus Buchungs-E-Mails stammen und damit zwangsläufig über den Server des Anbieters laufen.

3.3. Sprachlern-Apps

3.3.1. LingoDeer

LingoDeer ist eine kommerzielle Sprachlern-App mit Schwerpunkt auf ostasiatischen Sprachen wie Japanisch und Koreanisch, die strukturierte Lektionen, Vokabeln und Übungen bereitstellt und über einen Offline-Modus verfügt, für den die Lektionen vorab heruntergeladen werden [57]. Kostenlos sind lediglich die ersten Lektionen; der vollständige Zugang kostet 14,99 USD pro Monat beziehungsweise 95,99 USD pro Jahr [58]. Duolingo und Babbel, die in der Einleitung genannt wurden und die laut Umfrage am häufigsten genutzt werden, werden nicht gesondert betrachtet, da LingoDeer als Vertreter der abonnementbasierten Sprachlern-Apps mit seinem Sprachangebot näher am Zielsprachenumfang dieser Arbeit liegt.

3.3.2. Anki

Anki ist ein quelloffenes Karteikartenprogramm, das Vokabeln nach dem Prinzip der verteilten Wiederholung aus Abschnitt 2.2.2 abfragt. Die Desktop-Version ist kostenlos, die Karten werden lokal gespeichert, und der Abgleich zwischen Geräten über den Dienst AnkiWeb ist ein optionaler Zusatz [8]. Die iOS-Version AnkiMobile wird einmalig mit 29,99 EUR berechnet [59]. Vorgefertigte Lektionen enthält Anki nicht; Inhalte müssen selbst erstellt oder aus geteilten Kartensätzen importiert werden. Anki ist damit das Gegenstück zu LingoDeer: lokal und offen, aber ohne Inhalte.

3.4. Self-hosted-Lösungen

3.4.1. AdventureLog

Als Vertreter der self-hosted-Lösungen wurde AdventureLog aufgenommen, ein quelloffenes Reisetagebuch mit Reiseplanung und Weltkarte, das unter der GPL-3.0 steht und per Docker auf eigener Hardware betrieben wird [60]. Die Daten verbleiben damit auf dem eigenen Server. Als Webanwendung ist AdventureLog allerdings nur nutzbar, solange der Client den Server erreicht; einen Sprachlernbereich gibt es nicht. Architektonisch

entspricht AdventureLog dem zweiten Ansatz aus Abschnitt 2.5.5 und ist damit das Produkt, das dem Betriebsmodell des Prototyps am nächsten kommt.

Für den Vergleich fehlen noch die Kriterien.

3.5. Bewertungskriterien

Die Kriterien orientieren sich am Qualitätsmodell der ISO/IEC 25010 aus Abschnitt 2.3.1, beschränkt auf die Merkmale funktionale Eignung, Zuverlässigkeit und Sicherheit. Daraus wurden sechs Kriterien abgeleitet, die sich anhand der Herstellerangaben prüfen lassen. Die funktionale Eignung wird über drei Kriterien erfasst: den Umfang der Reisefunktionen (Reiseziele, Tagesplanung, Packliste, Budget), den Umfang der Sprachfunktionen (Vokabeln, Lektionen, Quiz, Fortschritt) und die Frage, ob beide Bereiche in einer Anwendung kombiniert sind. Die Offlinefähigkeit beschreibt, ob die Anwendung ohne Internetverbindung nutzbar ist, und betrifft damit die Zuverlässigkeit im Sinne der Verfügbarkeit. Die Datenhaltung erfasst, ob die Nutzerdaten lokal oder beim Anbieter gespeichert werden, und bildet die Vertraulichkeit aus dem Merkmal Sicherheit ab. Die Kosten sind kein Qualitätsmerkmal der Norm, für die Zielgruppe aber relevant und werden deshalb ergänzend aufgenommen.

Die Gebrauchstauglichkeit nach ISO 9241-11 (Abschnitt 2.3.2) lässt sich nur empirisch mit Nutzern beurteilen und ist deshalb nicht Teil dieses Vergleichs. Sie wird in Kapitel 7 für die Validierung des Prototyps herangezogen.

3.6. Vergleich

Tabelle 3.1 fasst die Bewertung der fünf Systeme nach den sechs Kriterien zusammen; die letzte Zeile zeigt zum Vergleich den angestrebten Prototyp.

Keines der betrachteten Systeme deckt beide Bereiche ab. Die Offlinefähigkeit ist bei den kommerziellen Produkten entweder an das kostenpflichtige Abonnement gebunden (Wanderlog, LingoDeer) oder auf den Abruf bereits angelegter Pläne beschränkt (TripIt). Eine lokale Datenhaltung bieten nur Anki und AdventureLog, also die beiden quelloffenen Anwendungen; die übrigen Produkte sind kontobasierte Cloud-Dienste, bei denen die Daten beim Anbieter liegen. Abbildung 3.1 ordnet die Systeme nach den beiden Dimensionen ein, die den Prototyp ausmachen.

3.7. Abgeleitete Lücke

Der Vergleich zeigt, dass Reiseplanung und Sprachenlernen am Markt getrennt bedient werden. Für die Reiseplanung gibt es Produkte, die aber entweder cloudgebunden sind

System	Reise	Sprache	Offline	Daten	Kosten
Wanderlog [55]	Ziele, Tagesplan, Budget	nein	nur Pro-Version	Anbieter-Cloud	gratis, Pro 39,99 USD/Jahr
TripIt [56]	Reiseplan aus Buchungen	nein	Abruf des Plans	Anbieter-Cloud	gratis, Pro 48,99 EUR/Jahr
Anki [8], [59]	nein	Vokabeln (Spaced Repetition)	vollständig	lokal, Sync optional	Desktop gratis, iOS 29,99 EUR
LingoDeer [57], [58]	nein	Lektionen, Vokabeln, Übungen	Offline-Modus, Lektionen vorab laden	Anbieter-Cloud	erste Lektionen gratis, 14,99 USD/Monat
AdventureLog [60]	Ziele, Tagesplan, Tagebuch	nein	nur im eigenen Netz	eigener Server	gratis (GPL-3.0), eigene Hardware
Prototyp dieser Arbeit	Ziele, Tagesplan, Packliste, Budget	Vokabeln, Lektionen, Quiz, Fortschritt	vollständig	Raspberry Pi	keine laufenden Kosten

Tabelle 3.1.: Kriteriengestützter Vergleich der betrachteten Systeme (Stand 2. November 2026)

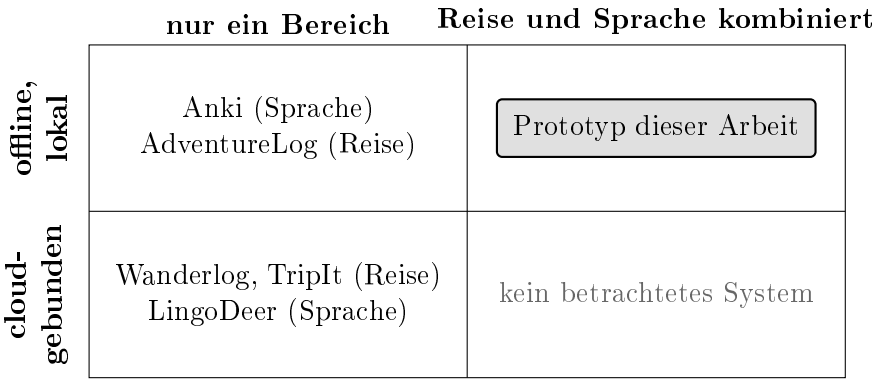


Abbildung 3.1.: Einordnung der betrachteten Systeme nach Funktionsumfang und Betriebsmodell (eigene Darstellung)

oder, wie AdventureLog, zwar lokal betrieben werden, jedoch keinen Sprachlernbereich bieten. Für das Sprachenlernen gilt das Umgekehrte: LingoDeer liefert fertige Lerninhalte, bindet sie aber an ein Abonnement und an die Infrastruktur des Anbieters; Anki arbeitet lokal und offline, bringt aber weder Lerninhalte noch einen Reisebezug mit.

Die Lücke liegt in der Kombination: beide Bereiche in einer Anwendung, vollständige Offlinefähigkeit, Datenhaltung auf eigener Hardware und keine laufenden Kosten gibt es jeweils einzeln, aber in keinem der Systeme zusammen. Lokale Datenhaltung findet

sich bei Anki und AdventureLog, der Funktionsumfang orientiert sich an Wanderlog und LingoDeer. Der Prototyp soll diese vier Eigenschaften zusammen erfüllen. Damit ist Aufgabe 1 bearbeitet. Ob potenzielle Nutzer diese Lücke ebenso sehen und welche Funktionen ihnen wichtig sind, klärt Kapitel 4.

4. Anforderungsanalyse

Dieses Kapitel wird derzeit bearbeitet.

4.1. Umfrage

4.1.1. Entwicklung des Fragebogens

4.1.2. Teilnehmer und Durchführung

4.1.3. Ergebnisse der geschlossenen Fragen

4.1.4. Freitextantworten

4.1.5. Limitationen

4.2. Use Cases

4.2.1. Akteur

4.2.2. Anwendungsfalldiagramm

4.2.3. Beschreibung der Anwendungsfälle

4.3. Funktionale Anforderungen

4.3.1. Reisebereich

4.3.2. Sprachbereich

4.3.3. Benutzer und Profile

4.4. Nichtfunktionale Anforderungen

4.5. Priorisierung (MoSCoW)

5. Konzeption

Dieses Kapitel wird derzeit bearbeitet.

5.1. Systemarchitektur

5.1.1. Schichten der Anwendung

5.1.2. Verteilung auf Raspberry Pi und Endgeräte

5.1.3. Technologieentscheidungen

5.2. Datenmodell

5.2.1. ER-Modell

5.2.2. Reisebereich

5.2.3. Sprachbereich

5.2.4. Lernfortschritt

5.3. Oberfläche

5.3.1. Navigationskonzept

5.3.2. Reise-Screens

5.3.3. Sprach-Screens

5.3.4. Mockups

5.4. Sicherheitskonzept

6. Implementierung

Dieses Kapitel folgt mit der Implementierung des Prototyps.

6.1. Projektstruktur und Build

6.2. Backend

6.2.1. Profilauswahl und Sitzung

6.2.2. Reise-Services

6.2.3. Wiederholungslogik

6.2.4. Quiz

6.3. Frontend

6.3.1. Templates

6.3.2. Bootstrap im Offline-Betrieb

6.4. Datenhaltung

6.4.1. Entities und Repositories

6.4.2. H2-Datenbankdatei

6.4.3. Laden der Sprachdaten

6.5. Betrieb auf dem Raspberry Pi

6.5.1. Betriebssystem und Java

6.5.2. Dienst und Autostart

6.5.3. Hostname und Erreichbarkeit

6.6. Automatisierte Tests

7. Evaluation

Dieses Kapitel folgt nach der Implementierung.

7.1. Verifikation

7.1.1. Unit-Tests

7.1.2. Funktionstests je Anwendungsfall

7.1.3. Datenbanktests

7.1.4. Offline-Test

7.1.5. Antwortzeiten

7.2. Validierung

7.2.1. Testdesign

7.2.2. Teilnehmer

7.2.3. Aufgaben

7.2.4. Ergebnis des System Usability Scale

7.2.5. Beobachtungen

7.3. Abgleich mit den Anforderungen

8. Kritische Reflexion

Dieses Kapitel folgt nach der Evaluation.

8.1. Methodik

8.2. Prototyp

8.3. Vorgehen und Zusammenarbeit

9. Ausblick

Dieses Kapitel folgt nach der Evaluation.

9.1. Funktionale Erweiterungen

9.2. Technische Erweiterungen

9.3. Weitere Untersuchungen

10. Methoden und Verfahren

Dieses Kapitel wird zum Abschluss der Arbeit vervollständigt.

10.1. Vorgehen

10.2. Eingesetzte Werkzeuge und Versionen

Tabelle 10.1 listet die verwendeten Werkzeuge und Bibliotheken mit den Versionen, die zum Abgabezeitpunkt in der `pom.xml` des Projekts stehen.

Werkzeug / Bibliothek	Version	Zweck
Java (OpenJDK)	21	Programmiersprache
Spring Boot	3.5.16	Backend-Framework
Thymeleaf	3.1.5	Server-seitiges Rendering
Bootstrap	5.3.8	Oberfläche
Bootstrap Icons	1.13.2	Icons
H2 Database	2.3.232	lokale Datenbank
Maven	3.10.0	Build

Tabelle 10.1.: Eingesetzte Werkzeuge und Bibliotheken mit Versionen (Stand 4. November 2026)

10.3. Nachvollziehbarkeit

Literatur

[1] Forschungsgemeinschaft Urlaub und Reisen e. V. (FUR), „Reiseanalyse 2026: Erste ausgewählte Ergebnisse der 56. Reiseanalyse," Forschungsgemeinschaft Urlaub und Reisen e. V., Kiel, 2026. besucht am 2. Nov. 2026. Adresse: https://reiseanalyse.de/wp-content/uploads/2026/02/RA2026_Erste_Ergebnisse_Broschuere_DE.pdf

[2] M. D. Falcón Falcón und P. A. Arias Arroyo, „Instrumental and Integrative Motivation in University Students," *Revista Scientific*, Jg. 9, Nr. 34, S. 22–40, 2024. DOI: 10.29394/Scientific.issn.2542-2987.2024.9.34.1.22-40

[3] Bitkom e. V. „Bürger nutzen im Schnitt 4 Reise-Apps für ihre Urlaubsplanung," besucht am 2. Nov. 2026. Adresse: https://www.bitkom.org/Presse/Presseinformation/Buerger-nutzen-im-Schnitt-4-Reise-Apps-fuer-ihre-Urlaubsplanung

[4] Wanderlog. „Wanderlog: Reiseplaner," besucht am 2. Nov. 2026. Adresse: https://wanderlog.com/

[5] TripIt. „TripIt: Travel Planner," besucht am 2. Nov. 2026. Adresse: https://www.tripit.com/

[6] Duolingo. „Duolingo," besucht am 2. Nov. 2026. Adresse: https://www.duolingo.com/

[7] Babbel. „Babbel," besucht am 2. Nov. 2026. Adresse: https://www.babbel.com/

[8] Anki. „Anki: Powerful, intelligent flash cards," besucht am 2. Nov. 2026. Adresse: https://apps.ankiweb.net/

[9] Bundesnetzagentur. „Sorgenfreies Roaming in den Sommerferien," besucht am 2. Nov. 2026. Adresse: https://www.bundesnetzagentur.de/SharedDocs/Pressemitteilungen/DE/2023/20230620_Roaming.html

[10] Verbraucherzentrale NRW. „Roaming in der EU: Keine Aufschläge – mit Ausnahmen," besucht am 2. Nov. 2026. Adresse: https://www.verbraucherzentrale.nrw/roaming

[11] S. Wexler, J. Shaffer und A. Cotgreave, *The Big Book of Dashboards: Visualizing Your Data Using Real-World Business Scenarios*. Hoboken: Wiley, 2017, ISBN: 978-1-119-28271-6.

[12] S. Few, *Information Dashboard Design: The Effective Visual Communication of Data*. Sebastopol: O'Reilly, 2006, ISBN: 978-0-596-10016-2.

[13] *ISO/IEC 25010:2023 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*, International Organization for Standardization, Genf, 2023. besucht am 2. Nov. 2026. Adresse: https://www.iso.org/standard/78176.html

[14] Europäisches Parlament und Rat. „Datenschutz-Grundverordnung: Verordnung (EU) 2016/679," besucht am 5. Nov. 2026. Adresse: https://eur-lex.europa.eu/eli/reg/2016/679/oj

[15] H. Ebbinghaus, *Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie*. Leipzig: Duncker & Humblot, 1885. besucht am 3. Nov. 2026. Adresse: https://archive.org/details/b28111710

[16] N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted und D. Rohrer, „Distributed practice in verbal recall tasks: A review and quantitative synthesis," *Psychological Bulletin*, Jg. 132, Nr. 3, S. 354–380, 2006. DOI: 10.1037/0033-2909.132.3.354

[17] S. Leitner, *So lernt man lernen: Angewandte Lernpsychologie – ein Weg zum Erfolg.* Augsburg: Weltbild, 1999, Lizenzausgabe der Herder-Ausgabe, Erstauflage 1972, ISBN: 3-89350-179-7.

[18] J. D. Karpicke und H. L. Roediger, „The Critical Importance of Retrieval for Learning," *Science*, Jg. 319, Nr. 5865, S. 966–968, 2008. DOI: 10.1126/science.1152408

[19] Council of Europe. „Common European Framework of Reference for Languages: Learning, teaching, assessment. Companion volume," besucht am 5. Nov. 2026. Adresse: https://rm.coe.int/common-european-framework-of-reference-for-languages-learning-teaching/16809ea0d4

[20] *ISO 9:1995 Information and documentation – Transliteration of Cyrillic characters into Latin characters – Slavic and non-Slavic languages*, International Organization for Standardization, Genf, 1995. besucht am 5. Nov. 2026. Adresse: https://www.iso.org/standard/3589.html

[21] Library of Congress. „ALA-LC Romanization Tables: Japanese," besucht am 5. Nov. 2026. Adresse: https://www.loc.gov/catdir/cpso/romanization/japanese.pdf

[22] National Institute of Korean Language. „Romanization of Korean," besucht am 5. Nov. 2026. Adresse: https://www.korean.go.kr/front_eng/roman/roman_01.do

[23] *DIN EN ISO 9241-11:2018-11 Ergonomie der Mensch-System-Interaktion – Teil 11: Gebrauchstauglichkeit: Begriffe und Konzepte (ISO 9241-11:2018)*, DIN Deutsches Institut für Normung e. V., Berlin: Beuth, 2018.

[24] J. Nielsen und R. Molich, „Heuristic evaluation of user interfaces," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '90)*, New York: ACM, 1990, S. 249–256. DOI: 10.1145/97243.97281

[25] J. Nielsen. „10 Usability Heuristics for User Interface Design," besucht am 1. Okt. 2026. Adresse: https://www.nngroup.com/articles/ten-usability-heuristics/

[26] J. Nielsen und T. K. Landauer, „A mathematical model of the finding of usability problems," in *Proceedings of the INTERACT '93 and CHI '93 Conference on Human Factors in Computing Systems*, New York: ACM, 1993, S. 206–213. DOI: 10.1145/169059.169166

[27] J. Nielsen. „Why You Only Need to Test with 5 Users," besucht am 5. Nov. 2026. Adresse: https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/

[28] J. Brooke, „SUS: A 'Quick and Dirty' Usability Scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester und I. L. McClelland, Hrsg., London: Taylor & Francis, 1996, S. 189–194. DOI: 10.1201/9781498710411-35

[29] A. Bangor, P. T. Kortum und J. T. Miller, „An Empirical Evaluation of the System Usability Scale," *International Journal of Human-Computer Interaction*, Jg. 24, Nr. 6, S. 574–594, 2008. DOI: 10.1080/10447310802205776

[30] A. Herrmann, *Grundlagen der Anforderungsanalyse: Standardkonformes Requirements Engineering*. Wiesbaden: Springer Vieweg, 2022. DOI: 10.1007/978-3-658-35460-2

[31] R. Likert, „A Technique for the Measurement of Attitudes," *Archives of Psychology*, Jg. 22, Nr. 140, S. 5–55, 1932.

[32] N. Döring und J. Bortz, *Forschungsmethoden und Evaluation in den Sozial- und Humanwissenschaften*, 5. Aufl. Berlin, Heidelberg: Springer, 2016. DOI: 10.1007/978-3-642-41089-5

[33] Agile Business Consortium. „What is MoSCoW Prioritization?" Besucht am 5. Nov. 2026. Adresse: https://www.agilebusiness.org/resource/what-is-moscow-prioritization/

[34] Object Management Group. „OMG Unified Modeling Language (OMG UML), Version 2.5.1," besucht am 5. Nov. 2026. Adresse: https://www.omg.org/spec/UML/2.5.1

[35] P. P.-S. Chen, „The entity-relationship model – toward a unified view of data,“ *ACM Transactions on Database Systems*, Jg. 1, Nr. 1, S. 9–36, 1976. DOI: 10.1145/320434.320440

[36] R. T. Fielding, M. Nottingham und J. Reschke. „RFC 9110: HTTP Semantics,“ besucht am 5. Nov. 2026. Adresse: https://www.rfc-editor.org/rfc/rfc9110

[37] Broadcom. „Spring Framework Reference: Spring Web MVC,“ besucht am 5. Nov. 2026. Adresse: https://docs.spring.io/spring-framework/reference/web/webmvc.html

[38] A. Barth. „RFC 6265: HTTP State Management Mechanism,“ besucht am 5. Nov. 2026. Adresse: https://www.rfc-editor.org/rfc/rfc6265

[39] Broadcom. „Spring Framework Reference: Classpath Scanning and Managed Components,“ besucht am 5. Nov. 2026. Adresse: https://docs.spring.io/spring-framework/reference/core/beans/classpath-scanning.html

[40] T. Ater, *Building Progressive Web Apps: Bringing the Power of Native to the Browser*. Sebastopol: O'Reilly, 2017, ISBN: 978-1-4919-6165-0.

[41] Mozilla Developer Network. „Progressive web apps,“ besucht am 3. Nov. 2026. Adresse: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

[42] S. Cheshire und M. Krochmal. „RFC 6762: Multicast DNS,“ besucht am 5. Nov. 2026. Adresse: https://www.rfc-editor.org/rfc/rfc6762

[43] Raspberry Pi Ltd. „Raspberry Pi Documentation: Remote access,“ besucht am 5. Nov. 2026. Adresse: https://www.raspberrypi.com/documentation/computers/remote-access.html

[44] Oracle. „JDK 21 (OpenJDK),“ besucht am 5. Nov. 2026. Adresse: https://openjdk.org/projects/jdk/21/

[45] Broadcom. „Spring Boot Reference Documentation,“ besucht am 3. Nov. 2026. Adresse: https://docs.spring.io/spring-boot/index.html

[46] The Thymeleaf Team. „Thymeleaf Documentation,“ besucht am 3. Nov. 2026. Adresse: https://www.thymeleaf.org/documentation.html

[47] Bootstrap. „Bootstrap 5.3 Documentation: Introduction,“ besucht am 5. Nov. 2026. Adresse: https://getbootstrap.com/docs/5.3/getting-started/introduction/

[48] WebJars. „WebJars: Web Libraries in Jars,“ besucht am 5. Nov. 2026. Adresse: https://www.webjars.org/

[49] H2 Group. „H2 Database Engine,“ besucht am 3. Nov. 2026. Adresse: https://www.h2database.com/html/main.html

[50] Eclipse Foundation. „Jakarta Persistence 3.1 Specification,“ besucht am 5. Nov. 2026. Adresse: https://jakarta.ee/specifications/persistence/3.1/

[51] Red Hat. „Hibernate ORM Documentation,“ besucht am 5. Nov. 2026. Adresse: https://hibernate.org/orm/documentation/

[52] Raspberry Pi Ltd. „Raspberry Pi 5: Specification,“ besucht am 3. Nov. 2026. Adresse: https://www.raspberrypi.com/products/raspberry-pi-5/

[53] Apache Software Foundation. „Apache Maven Project,“ besucht am 5. Nov. 2026. Adresse: https://maven.apache.org/

[54] Software Freedom Conservancy. „Git,“ besucht am 5. Nov. 2026. Adresse: https://git-scm.com/

[55] Wanderlog. „Wanderlog Pro: Preisvergleich und Funktionen,“ besucht am 2. Nov. 2026. Adresse: https://wanderlog.com/pro

[56] TripIt. „TripIt: Travel Planner. App Store Deutschland,“ besucht am 2. Nov. 2026. Adresse: https://apps.apple.com/de/app/tripit-travel-planner/id311035142

[57] LingoDeer. „LingoDeer: Sprachen lernen. App Store Deutschland,“ besucht am 2. Nov. 2026. Adresse: https://apps.apple.com/de/app/lingodeer-sprachen-lernen/id1261193709

[58] LingoDeer. „LingoDeer Premium: Preise,“ besucht am 2. Nov. 2026. Adresse: https://www.lingodeer.com/de/pricing

[59] Anki. „AnkiMobile Flashcards. App Store Deutschland,“ besucht am 2. Nov. 2026. Adresse: https://apps.apple.com/de/app/ankimobile-flashcards/id373493387

[60] S. Morley. „AdventureLog: Self-hosted travel companion,“ besucht am 2. Nov. 2026. Adresse: https://adventurelog.app/

Anmerkung zur Nutzung von Künstlicher Intelligenz

Bei der Anfertigung dieser Arbeit wurden die KI-gestützten Sprachmodelle Claude (Anthropic) und ChatGPT (OpenAI) als Hilfsmittel eingesetzt. Sie dienten der sprachlichen Formulierung und Überarbeitung des Textes, der Strukturierung von Abschnitten, der Unterstützung beim Satz in LaTeX und bei der technischen Umsetzung der Abbildungen. Tabelle K1 fasst die Nutzung zusammen.

Die inhaltliche Substanz der Arbeit stammt von den Verfassern: Das Dashboard wurde eigenständig konzipiert und implementiert, die Nutzerumfrage selbst durchgeführt und ausgewertet, und alle technischen Entscheidungen wurden selbst getroffen. Sämtliche fachlichen Aussagen, Quellen und Ergebnisse wurden eigenständig geprüft und werden inhaltlich verantwortet. KI-gestützte Werkzeuge wurden nicht zur Erzeugung von Daten oder Ergebnissen verwendet.

Werkzeug	Beschreibung der Nutzung
Claude (Anthropic)	Sprachliche Formulierung und Überarbeitung (gesamt), Strukturierung von Abschnitten, Hilfe beim LaTeX-Satz und bei den Abbildungen
ChatGPT (OpenAI)	Sprachliche Überarbeitung und Verständnisfragen zu Grundbegriffen (gesamt)

Tabelle K1.: Übersicht über die verwendeten KI-basierten Werkzeuge

A. Fragebogen der Online-Umfrage

Die Umfrage wurde mit Microsoft Forms erstellt und vom 9. bis 11. Oktober 2026 anonym durchgeführt. Die Skalenfragen verwenden fünf Stufen von 1 (unwichtig) bis 5 (sehr wichtig).

1. Wie oft reist du ins Ausland? (Einfachauswahl: Mehrmals im Jahr, Etwa 1x im Jahr, Selten)
2. Lernst du vor oder auf einer Reise etwas von der Landessprache? (Einfachauswahl: Immer, Manchmal, Nie)
3. Welche Apps nutzt du für die Reiseplanung? (Mehrfachauswahl: Google Maps, Wanderlog, Notizen-App, Keine, Sonstiges mit Freitext)
4. Welche Apps nutzt du zum Sprachenlernen? (Mehrfachauswahl: Duolingo, Babbel, Anki, Keine, Sonstiges mit Freitext)
5. Was stört dich an den aktuellen Lösungen? (Freitext)
6. Wie wichtig sind dir die folgenden Funktionen? (Skala 1 bis 5 je Funktion: Reiseziele, Tagespläne, Packlisten, Budget, Notizen, Vokabeln, Reisephrasen, Quiz, Lernfortschritt)
7. Wie wichtig ist dir, dass die App ohne Internet funktioniert? (Skala 1 bis 5)
8. Wie wichtig ist dir, dass deine Daten auf deinem Gerät bleiben, nicht auf einem Server? (Skala 1 bis 5)
9. Würdest du so eine App in deinem Alltag nutzen, um dich auf die nächste Reise vorzubereiten? (Einfachauswahl: Ja, Vielleicht, Nein)
10. Welche Funktionen sollte diese App unbedingt haben? (Freitext)
11. Für welche Länder würdest du dir so eine App insbesondere wünschen? (Freitext)
12. Welche Altersgruppe? (Einfachauswahl: 18–25, 26–35, 36–50, über 50)

B. Rohdaten der Umfrage

Die Tabellen B.1 und B.2 enthalten die anonymisierten Antworten der zwölf Teilnehmenden aus dem Export der Umfrage (Datei `Umfrage_Rohdaten_1-12.xlsx`). Die Spalten der Skalenfragen sind abgekürzt: RZ Reiseziele, TP Tagespläne, PL Packlisten, BU Budget, NO Notizen, VO Vokabeln, RP Reisephrasen, QZ Quiz, LF Lernfortschritt, OF Offline-Nutzung, DL Daten lokal.

ID	Reisen	Lernen	RZ	TP	PL	BU	NO	VO	RP	QZ	LF	OF	DL	Nutzen	Alter
1	mehrmals/Jahr	immer	4	3	5	4	4	3	3	4	4	4	3	ja	26–35
2	mehrmals/Jahr	nie	5	4	3	5	5	3	3	4	5	4	3	vielleicht	18–25
3	mehrmals/Jahr	immer	3	4	5	1	1	3	3	1	4	5	5	vielleicht	26–35
4	mehrmals/Jahr	nie	4	4	4	4	2	3	1	4	5	1	1	ja	18–25
5	selten	manchmal	4	3	3	5	3	3	3	2	2	5	3	vielleicht	18–25
6	etwa 1x/Jahr	manchmal	5	5	4	4	3	5	5	2	2	5	4	ja	36–50
7	mehrmals/Jahr	manchmal	3	3	3	3	3	3	3	3	3	3	3	nein	über 50
8	mehrmals/Jahr	manchmal	4	3	3	2	3	4	5	3	3	5	3	ja	18–25
9	mehrmals/Jahr	nie	5	4	5	5	2	1	1	1	1	3	5	vielleicht	18–25
10	etwa 1x/Jahr	manchmal	5	5	3	4	3	3	4	2	3	5	3	ja	18–25
11	mehrmals/Jahr	manchmal	5	1	3	1	1	1	1	1	1	5	4	vielleicht	über 50
12	mehrmals/Jahr	immer	4	3	2	3	3	5	5	3	4	5	4	ja	18–25

Tabelle B.1.: Antworten auf die geschlossenen Fragen

ID	Frage	Antwort
1	Apps	Reise: Google Maps, Notizen-App. Sprache: Duolingo
1	Kritik	Zu viele Aufforderungen (Push-Benachrichtigungen) zum Lernen
2	Apps	Reise: Google Maps, Notizen-App, Gemini/Claude. Sprache: Duolingo
2	Wunsch	Soziale Funktionen (Freunde, etc.)
3	Apps	Reise: keine. Sprache: Babbel
3	Wunsch	Funktion andere Seiten einzubinden oder Dateien hochzuladen wie Reisepläne
4	Apps	Reise: Notizen-App. Sprache: Duolingo, Anki
4	Kritik	Entweder Kosten, keine richtige Grammatik oder dafür Grammatik aber derbe trocken
4	Wunsch	Gemeinsame Ziele mit anderen planen zu können
4	Länder	Vor allem für die Länder um Deutschland
5	Apps	Reise: Google Maps, Notizen-App. Sprache: Duolingo
6	Apps	Reise: Google Maps. Sprache: Duolingo
7	Apps	Reise: Notizen-App, diverse. Sprache: keine
7	Kritik	Internationale Bahntickets kaufen
7	Wunsch	Das Umfragedesign zwingt bei unbeantwortbaren Fragen zu falschen Antworten, außerdem ist Frage 6 unsinnig
8	Apps	Reise: Google Maps, Notizen-App. Sprache: Duolingo, Babbel
8	Kritik	Dass alles in verschiedenen Apps verstreut ist. Reiseplan in der einen, Sprache in der anderen, und offline geht Duolingo kaum. Nervt, dass man ständig wechselt
8	Wunsch	Offline-Karten und Vokabeln an einem Ort, die wichtigsten Sätze für das Land schon vorbereitet, und dass es ohne Account läuft
8	Länder	Italien, Spanien, und was Exotischeres wie Japan oder Georgien, wo die Sprache echt eine Hürde ist
9	Apps	Reise: Google Maps, Notizen-App, TikTok, GetYourGuide. Sprache: keine
10	Apps	Reise: Google Maps, Wanderlog, Notizen-App. Sprache: Duolingo, Anki
10	Kritik	Die Grammatik ist manchmal falsch
10	Länder	Korea, Japan, Thailand, China, Spanien, Italien
11	Apps	Reise: OruxMaps. Sprache: keine
11	Kritik	Nichts
11	Länder	Frankreich, Norwegen
12	Apps	Reise: Google Maps, Notizen-App. Sprache: Duolingo, Anki
12	Kritik	Man hat ständig zwei Apps parallel offen, Reiseplanung in der einen und Vokabeln in der anderen. Und Duolingo bringt mir für Russisch oder Japanisch kaum was Praktisches, zu viel Spielkram und zu wenig Sätze, die man unterwegs echt braucht
12	Wunsch	Offline-Modus auf jeden Fall, in China und Russland ist das Netz manchmal Mist. Und Reisephrasen nach Situation sortiert, also Flughafen, Restaurant, Taxi. Nicht nur random Vokabeln
12	Länder	Russland, Japan und China. Gerade für die drei gibt es kaum was Gutes, die meisten Apps können nur die Standard-Europa-Sprachen richtig

Tabelle B.2.: Genutzte Apps und Freitextantworten (Rechtschreibung behutsam angeglichen, Inhalt unverändert)